

Proyecto coder SQL

Link del repositorio: En GitHub, podrán encontrar las consultas SQL para crear y rellenar la base de datos, así como para definir los triggers, vistas y procedimientos almacenados:

<https://github.com/sebamarambio96/sql-coder>

Introducción: En el contexto de la gestión de servicios médicos, es fundamental contar con un sistema eficiente que permita registrar y administrar tanto la información de los pacientes como la disponibilidad de citas.

Objetivo: Desarrollar un sistema debe registrar:

- Información de cada persona relacionada al proceso.
- Información de los profesionales.
- Horarios de la sesión.
- Detalles de la sesión.
- Confirmación tanto de pacientes y doctores.
- Manejo permisos dentro del sistema.

Situación problemática: Me gustaría desarrollar un sistema donde mi esposa pueda registrar las fichas de sus pacientes y donde los pacientes puedan reservar sus citas disponibles.

Modelo de negocio

Person

Registra los datos de las personas para ser asociados a distintas tablas como doctor y usuario, su objetivo es guardar la información personal de las personas pertenecientes al sistema.

Columna	Tipo	Restricciones	Descripción
id	integer	PRIMARY KEY	Identificador único de la persona
name	varchar(255)		Nombre completo de la persona
dni	integer	UNIQUE	Documento Nacional de Identidad
dni_verification_digit	integer		Dígito de verificación del DNI en caso de ser (RUT)
age	integer		Edad de la persona
email	varchar(255)		Correo electrónico de la persona
created_at	timestamp		Fecha y hora de creación del registro

Doctor

Registra el personal medico que podría estar disponible.

Columna	Tipo	Restricciones	Descripción
id	integer	PRIMARY KEY	Identificador único del doctor
person_id	integer	FOREIGN KEY (person_id) REFERENCES person(id)	Identificador de la persona asociado
specialty_id	integer	FOREIGN KEY (specialty_id) REFERENCES specialty(id)	Identificador de la especialidad del doctor
email	varchar(255)		Correo profesional del doctor
phone_number	varchar(20)		Número de teléfono profesional del doctor
created_at	timestamp		Fecha y hora de creación del registro

Specialty

Registra los tipos de especialidades disponibles en el centro medico

Columna	Tipo	Restricciones	Descripción
id	integer	PRIMARY KEY	Identificador único de la especialidad
name	varchar(255)	UNIQUE	Nombre de la especialidad
description	text		Descripción de la especialidad
created_at	timestamp		Fecha y hora de creación del registro

User

Registra las credenciales usadas en el sistema

Columna	Tipo	Restricciones	Descripción
id	integer	PRIMARY KEY	Identificador único de las credenciales de usuario
person_id	integer	FOREIGN KEY (person_id) REFERENCES person(id)	Identificador de la persona asociada
username	varchar		Nombre de usuario
pass	varchar		Contraseña del usuario
role	varchar	FOREIGN KEY (role) REFERENCES	Rol del usuario en el sistema

		user_role(role_name)	
created_at	timestamp		Fecha y hora de creación del registro

User_role

Registra los tipos de roles de usuarios disponibles como: paciente, doctor, admin,etc

Columna	Tipo	Restricciones	Descripción
role_name	varchar(255)	PRIMARY KEY	Nombre del rol
description	text		Describe los alcances del rol en el sistema
created_at	timestamp		Fecha y hora de creación del registro

Session

Registra las horas disponibles de sesiones medicas.

Columna	Tipo	Restricciones	Descripción
id	integer	PRIMARY KEY	Identificador único de la sesión
doctor_id	integer	FOREIGN KEY (doctor_id) REFERENCES doctor(id)	Identificador del doctor asociado a la cita
start_time	timestamp		Hora de inicio de la cita
end_time	timestamp		Hora de finalización de la cita
confirmed_by_doctor	boolean	DEFAULT: false	Indica si la cita ha sido confirmada por el doctor
available	boolean	DEFAULT: false	Indica si la cita está disponible
created_at	timestamp		Fecha y hora de creación del registro

Session_request

Registra las peticiones de sesiones por parte de pacientes.

Columna	Tipo	Restricciones	Descripción
id	integer	PRIMARY KEY	Identificador único de la petición de sesión

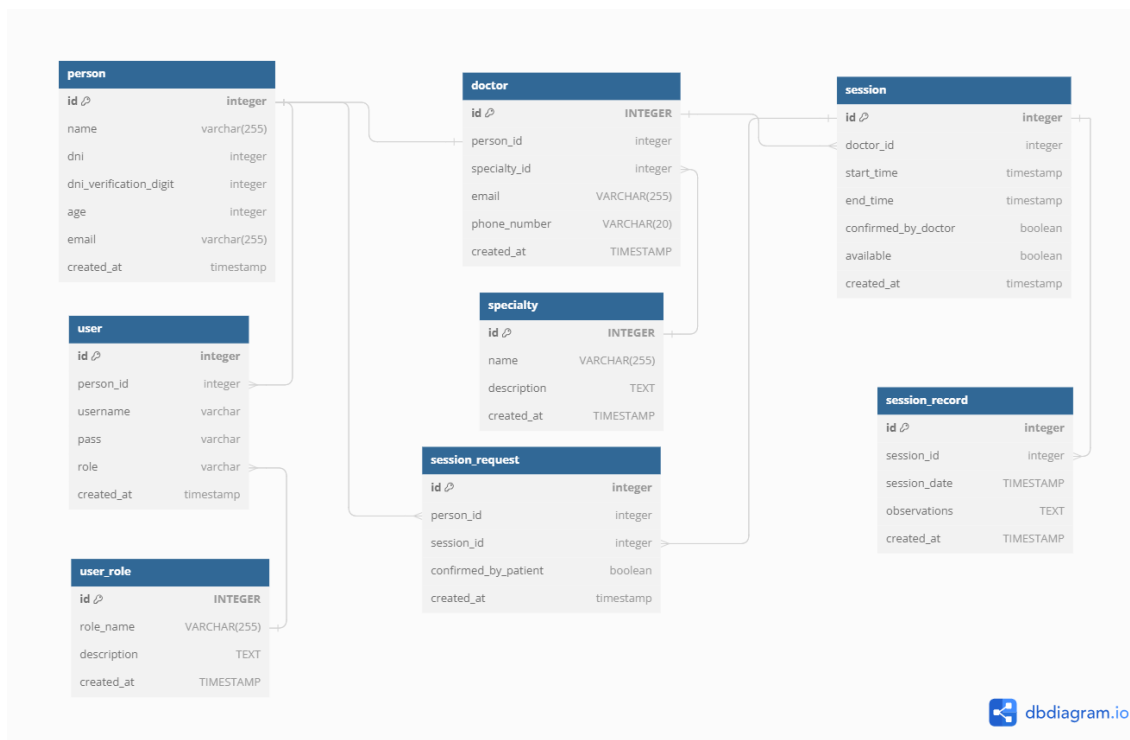
person_id	integer	FOREIGN KEY (person_id) REFERENCES person(id)	Identificador de la persona asociada (paciente)
session_id	integer	FOREIGN KEY (session_id) REFERENCES session(id)	Identificador de la sesión solicitada
confirmed_by_patient	boolean	DEFAULT: false	Indica si la petición de sesión ha sido confirmada por el paciente
created_at	timestamp		Fecha y hora de creación del registro

Session_record

Registra la ficha medica, como detalles de la sesion y anotaciones del medico.

Columna	Tipo	Restricciones	Descripción
id	integer	PRIMARY KEY	Identificador único de la sesión
session_id	integer	FOREIGN KEY (session_id) REFERENCES session(id)	Identificador de la sesión relacionada
session_date	timestamp		Fecha y hora de la sesión
observations	text		Observaciones realizadas durante la sesión
created_at	timestamp		Fecha y hora de creación del registro

Diagrama E-R



Funciones

GET_PATIENT_SESSION_REQUESTS

Calcula el número de solicitudes de sesiones realizadas por un paciente específico.

```

DELIMITER //
CREATE FUNCTION get_patient_session_requests(p_person_id INTEGER)
RETURNS INTEGER
BEGIN
    DECLARE session_requests INTEGER;

    SELECT COUNT(*) INTO session_requests FROM session_request WHERE person_id =
    p_person_id;

    RETURN session_requests;
END//
  
```

GET_SESSION_COUNT

Calcula la cantidad de sesiones asignadas a un doctor específico.

```

DELIMITER //
CREATE FUNCTION get_session_count(p_doctor_id INTEGER)
RETURNS INTEGER
BEGIN
    DECLARE session_count INTEGER;

    SELECT COUNT(*) INTO session_count FROM session WHERE doctor_id = p_doctor_id;

    RETURN session_count;
END//
  
```

Vistas

PATIENT_SESSION_REQUESTS

Muestra el número de solicitudes de sesiones realizadas por cada paciente.

```
CREATE VIEW patient_session_requests AS
SELECT person.id AS person_id, get_patient_session_requests(person.id) AS
session_requests
FROM person;
```

DOCTOR_COUNT_SESSIONS

Muestra la cantidad de sesiones asignadas a cada doctor junto con el nombre del doctor.

```
CREATE VIEW doctor_count_sessions AS
SELECT
    d.id AS doctor_id,
    p.name AS doctor_name,
    get_session_count(d.id) AS session_count
FROM
    doctor d
JOIN
    person p ON d.person_id = p.id;
```

USER_ROLES

Muestra los roles y detalles de los usuarios.

```
CREATE VIEW user_roles AS
SELECT
    u.id AS user_id,
    p.name AS user_name,
    u.username,
    ur.role_name,
    ur.description AS role_description,
    u.created_at AS user_created_at
FROM
    user u
JOIN
    person p ON u.person_id = p.id
JOIN
    user_role ur ON u.role = ur.role_name;
```

DOCTOR_SESSIONS

Muestra las sesiones programadas para cada doctor.

```
CREATE VIEW doctor_sessions AS
SELECT
    d.id AS doctor_id,
    p.name AS doctor_name,
    s.id AS session_id,
    s.start_time,
    s.end_time,
```

```

        s.confirmed_by_doctor,
        s.available,
        s.created_at AS session_created_at
FROM
    doctor d
JOIN
    person p ON d.person_id = p.id
JOIN
    session s ON d.id = s.doctor_id;

```

Procedimientos almacenados

CONFIRM_BY_PATIENT

Aprueba una solicitud de sesión por parte del paciente.

```

DELIMITER //

CREATE PROCEDURE confirm_by_patient(
    IN p_request_id INTEGER
)
BEGIN
    UPDATE session_request
    SET confirmed_by_patient = TRUE
    WHERE id = p_request_id;
END//

```

CREATE_SESSION

Crea una nueva sesión en la base de datos.

```

DELIMITER //

CREATE PROCEDURE confirm_by_patient(
    IN p_request_id INTEGER
)
BEGIN
    UPDATE session_request
    SET confirmed_by_patient = TRUE
    WHERE id = p_request_id;
END//

```

Triggers

CREATE_SESSION_RECORD

Registra un nuevo registro de sesión cada vez que se inserta un nuevo registro en la tabla session.

```

DELIMITER //

CREATE TRIGGER create_session_record AFTER INSERT ON session
FOR EACH ROW
BEGIN
    INSERT INTO session_record (session_id, session_date, observations, created_at)

```

```
VALUES (NEW.id, NEW.start_time, 'Ingreso observación aquí', CURRENT_TIMESTAMP);  
END//
```

CREATE_USER_AFTER_PERSON_INSERT

Crea un nuevo usuario cuando se inserta un nuevo registro en la tabla person.

```
DELIMITER //  
  
CREATE TRIGGER create_user_after_person_insert AFTER INSERT ON person  
FOR EACH ROW  
BEGIN  
    INSERT INTO user (person_id, username, pass, role, created_at)  
    VALUES (NEW.id, CONCAT('user_', NEW.id), 'default_password', 'paciente',  
CURRENT_TIMESTAMP);  
END//
```

Seeders

En la creación de la tabla se incluyen datos a insertar pero de todas maneras para practicar he creado archivos JSON para importar datos.

A continuación detallaré como importar datos de los archivos JSON que adjuntaré a la documentación del proyecto.

Instrucciones

1. Abrir worbench, seleccionar una tabla y abrir la herramienta "Import record from external file".
2. Encontrar el archivo JSON que contiene la información de la tabla seleccionada.
3. Ahora seleccionar la tabla y seleccionar tabla existente.
4. Luego apretar "Next" hasta que se genere la importación, revisar que se hayan insertado 10 rows.

A continuación el orden en el que deben importarse los archivos:

1. SpecialtySeeder
2. UserRoleSeeder
3. PersonSeeder
4. DoctorSeeder
5. UserSeeder
6. SessionSeeder
7. SessionRequestSeeder
8. SessionRecordSeeder

Considerar: Aplicar estas importaciones antes de configurar los triggers.

Herramientas y Tecnologías Utilizadas

1. Sistema de Gestión de Bases de Datos (DBMS)
 - MySQL: Para el almacenamiento y gestión de datos de pacientes, doctores, sesiones y otros.
2. Lenguaje de Consulta SQL

- SQL (Structured Query Language): Para definir y manipular la base de datos, crear tablas, vistas, funciones, procedimientos almacenados y triggers.

3. Entorno de Desarrollo

- MySQL Workbench: Herramientas de interfaz gráfica para la administración y desarrollo de la base de datos.

4. Sistema de Control de Versiones

- Git: Para el control de versiones del código y gestión de cambios.
- GitHub: Para alojar el repositorio del proyecto y colaborar con otros desarrolladores.

5. Documentación

- Markdown: Para la redacción de documentación técnica y descripciones del sistema.
- dbdiagram.io: Para la creación del diagrama E - R